

作業系統 期中考 練習考卷

姓名：_____ 學號：_____ 日期：_____

本試卷共五大題，滿分 100 分。作答時間 90 分鐘。程式碼填充題中，_____ 代表需要填入的內容；已給的程式碼請勿更動。

第一題 中斷與 I/O 機制 (20 分)

(1) Interrupt-driven I/O 流程排列 (10 分)

下列是中斷驅動 I/O 循環的七個步驟，請按正確執行順序填入右方空格，並於下方說明各步驟的功能。

- (i) Input ready, output complete, or error generates interrupt signal

(ii) Interrupt handler processes data, return from interrupt

(iii) CPU executing, checks for interrupts between instructions

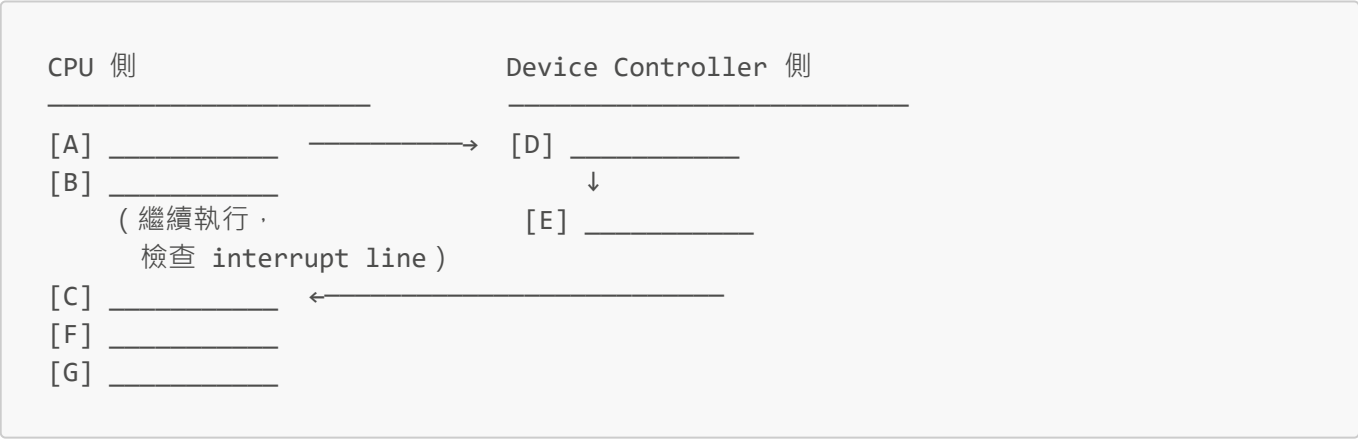
(iv) Device driver initiates I/O

(v) Controller initiates I/O

(vi) CPU resumes processing of interrupted process

(vii) CPU receiving interrupt, transfer control to interrupt handler

請填入 CPU 側與 Device Controller 側的執行順序：



(2) Interrupt-based 與 Busy-waiting 比較 (5 分)

請說明 interrupt-based I/O 相對於 busy-waiting 的優點。

答：

(3) Device Controller 與 Device Driver 的角色 (5 分)

請分別說明 Device Controller 和 Device Driver 在電腦系統中的功能。

Device Controller :

Device Driver :

第二題 系統結構 (20 分)

(1) API 如何呼叫 System Call (10 分)

請說明 API 函數如何呼叫系統呼叫 (system call) 。 提示：你的答案必須包含以下四個關鍵字：`software interrupt`、`dual-mode`、`system call number (index)`、`table of code pointer`

答：

(2) OS 結構比較 (10 分)

(a) 請說明 **Layered OS** 架構的一個優點與一個缺點。(4 分)

優點：

缺點：

(b) 請說明 **Microkernel** 架構的一個優點與一個缺點。(4 分)

優點：

缺點：

(c) **Modular Kernel** 相比 **Microkernel**，在效能上有何優勢？（2 分）

答：

第三題 行程（Processes）（20 分）

(1) 行程狀態判斷（8 分）

請填入下列各情境中，行程（process）所處的狀態（new / ready / running / waiting / terminated）：

#	情境	行程狀態
a	行程被 long-term scheduler 選中，載入記憶體後	_____
b	行程等待被 short-term scheduler 選中執行前	_____
c	行程被 short-term scheduler 選中，開始使用 CPU	_____
d	行程呼叫 blocking system call 之前	_____
e	行程呼叫 blocking system call 之後	_____
f	Blocking system call 完成之後	_____
g	行程被 hardware interrupt 中斷後	_____
h	行程的 time slice 用盡（time quantum expired）後	_____

(2) fork() 輸出分析（12 分）

請分析下列程式，回答問題：

```
int value = 5; /* value is a global variable */

int main(int argc, char *argv[]) {
    pid_t pid;
    pid = fork();

    if (pid > 0) {
        value += 15;
        wait(NULL);
        printf("value = %d\n", value);
    } else if (pid == 0) {
        value += 5;
        printf("value = %d\n", value);
    }
}
```

```
        return 0;
    }
```

(a) 請寫出程式的輸出結果（順序必須正確）：（6分）

輸出：

（說明各行是哪個行程印出的）

(b) 若將 `wait(NULL)` 這行移除，對程式執行有何影響？（6分）

影響一（執行順序）：

影響二（孤兒行程）：

影響三（殭屍行程）：

第四題 執行緒（Threads）（20分）

(1) Thread Model 比較（9分）

請說明下列三種執行緒模型的優缺點，並各舉一個例子（OS 或系統）：

模型	優點	缺點	例子
Many-to-One			
One-to-One			
Many-to-Many			

(2) Scheduler Activations 機制（6分）

(a) 什麼是 **upcall**？它在 Many-to-Many 模型中扮演什麼角色？（3分）

答：

(b) 當一個 user-level thread 即將執行 blocking system call 時，Scheduler Activations 機制會如何處理，以避免整個 process 被 block？(3 分)

答：

(3) LWP 數量 (5 分)

請說明在 Many-to-Many 模型中，適當的 LWP (Lightweight Process) 數量應如何決定？請分別說明 CPU-bound 與 I/O-bound 程式的建議數量及原因。

CPU-bound 程式：

I/O-bound 程式：

第五題 程式碼填充 (20 分)

(1) Shared Memory 計算程式 (10 分)

下列程式使用 **shared memory** 讓子行程協助計算 $2x + 3y - 5$ 。
請填入 (A) 到 (H) 各空格的內容，使程式正確運行。

```
#include <sys/shm.h>
#include <sys/stat.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);
    int i;
    pid_t pid;
    const int sh_size = 4;

    /* 建立共享記憶體區段 */
    int segment_id = ____ (A) ____ (IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);

    /* 將共享記憶體附加到此行程的位址空間 */
```

```

int *shared_memory = (int *) ____ (B) ____ (segment_id, NULL, 0);

/* 初始化共享記憶體 */
*shared_memory = ____ (C) ____;

for (i = 2; i > 0; i--) {
    pid = ____ (D) ____ ();
    if (pid < 0) {
        fprintf(stderr, "Fork Failed");
        exit(-1);
    } else if (pid == 0) {          /* 子行程 */
        if (i == 2) *shared_memory += ____ (E) ____;    /* 計算 2x */
        if (i == 1) *shared_memory += ____ (F) ____;    /* 計算 3y */
        shmdt(shared_memory);
        exit(0);
    } else {                      /* 父行程 */
        ____ (G) ____ (NULL);      /* 等待子行程，避免 race condition */
    }
}

*shared_memory -= 5;             /* 計算 -5 */
printf("the result is %d\n", *shared_memory);

/* 分離並釋放共享記憶體 */
shmdt(shared_memory);
____ (H) ____ (segment_id, IPC_RMID, NULL);
return 0;
}

```

空格 填入內容

(A)

(B)

(C)

(D)

(E)

(F)

(G)

(H)

(2) Shell Interface 程式 (10 分)

下列 C++ 程式實作一個簡單的 shell，可接受使用者命令並在子行程中執行。若命令結尾有 `&`，則在背景執行（父行程不等待）。請填入 (A) 到 (F) 各空格的内容，使程式正確運行。

```

#include <iostream>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
using namespace std;

int main(void) {
    char inputBuffer[80];
    bool background;

    while (cin >> inputBuffer) {
        cout << endl << " COMMAND->";

        pid_t pid;

        /* Step 1: 檢查命令是否以 '&' 結尾 */
        if (inputBuffer[strlen(inputBuffer)-1] == '&') {
            background = ____(A)__;
            inputBuffer[strlen(inputBuffer)-1] = '\\0'; /* 移除 '&' */
        } else {
            background = ____(B)__;
        }

        /* Step 2: 建立子行程 */
        pid = ____(C)____();

        if (pid == 0) {
            /* 子行程：執行使用者輸入的命令 */
            ____(D)____(inputBuffer);
        } else if (pid > 0) {
            /* 父行程：依 background 決定是否等待 */
            if (background == ____(E)____)
                ____(F)____(NULL); /* 前景執行：等子行程結束 */
            /* 背景執行：直接繼續，不等待 */
        }

    } /* end of while */
    return 0;
} /* end of main */

```

空格 填入內容

(A)

(B)

(C)

(D)

(E)

(F)

附錄：CPU 排程補充題 (選答，10 分加分)

多層回饋佇列 (Multilevel Feedback Queue) 排程

有以下行程，使用三層 Multilevel Feedback Queue (MFQ) 排程：

- **Q2** (最高優先)：RR，time quantum = 4 ms
- **Q1** (中間)：RR，time quantum = 6 ms
- **Q0** (最低)：FCFS

行程	到達時間	CPU Burst
P1	0	8 ms
P2	2	4 ms
P3	4	12 ms
P4	6	6 ms

(a) 請畫出 Gantt Chart：

時間軸 (ms)：

024681012141618202224262830

[][][][][][][][][][][][][][]

說明各行程在哪個 queue 執行：

(b) 請計算每個行程的 waiting time 與平均 waiting time：

行程	到達時間	完成時間	Turnaround Time	Waiting Time
P1	0			
P2	2			
P3	4			
P4	6			

平均 Waiting Time = _____ ms

試卷結束，祝考試順利！